

01 / THE QUESTION

What are we trying to improve?

A plain-language explanation of the project, using our completed A100 experiments. Evidence recorded 8 September 2026.

Inference is using an already trained model to produce an answer. A user sends a prompt, the model processes it, and the answer appears a small piece at a time. These pieces are called **tokens**; a token can be a word, part of a word or punctuation. The GPU performs the model's numerical calculations.

Why one request can affect another

A text-generating model first processes the input (**prefill**), then produces output tokens step by step (**decode**). A long document and a short question can put different demands on these stages. A serving engine groups requests into batches and shares GPU time between them. Decisions that improve total throughput can also delay individual responses. [\[1\]](#)

Imagine 96 short questions arriving, then 48 long documents, followed by 96 more short questions. Our experiment uses that length pattern. During the long-prompt phase, each request brings **16 times as many input tokens**, although this does not imply 16 times the processing time. If work arrives faster than it finishes, a backlog builds. Switching back to short prompts does not instantly clear that backlog: a new short request may spend its entire latency allowance waiting for earlier work.

Can inference scheduling complete more requests within latency targets on the same GPU when a burst of long prompts delays subsequent short requests?

What “more useful GPU work” means here

The goal is to get more timely answers from the same hardware. **Admission** decides when a waiting request enters the engine. **Scheduling** decides how admitted work shares execution. Our first candidate changes admission outside an existing engine; it does not replace all of the engine's scheduling logic.

Throughput counts work completed per second, such as output tokens/s. **Latency** is the time a user waits. A **service-level objective (SLO)** is a declared latency target; a request must meet our targets to count toward timely completed work.

A GPU reporting 99% utilization may simply have a kernel (a GPU program) running almost all the time. Two schedules can keep it active for the same second yet finish different amounts of work, or make different users wait. Busy time therefore cannot tell us which schedule produces more timely answers or uses the processing units more efficiently. We measure completed work and latency alongside it. [\[2\]](#)

The hypothesis remains a question

Perhaps estimates of processing cost and time remaining before a deadline can guide better admission decisions. Equally, an extra gate may hold back useful work and make performance worse. Our measurements must distinguish those possibilities; an improvement is not assumed.

02 / SYSTEM AND RATIONALE

What did we run, and why?

Two separate studies test different decisions in the same inference stack. Neither changes the trained model weights.

Component	Recorded system
GPU	One NVIDIA A100 PCIe 40 GB (40,960 MiB); 250 W power limit; driver 595.58.03
Software	Python 3.12.13; vLLM 0.28.0; PyTorch 2.13.0; Transformers 5.16.1
Model execution	BF16 precision; tensor parallelism 1 (the model runs on one GPU); text-only use
Input material	Real WikiText-2 test text, sliced into exact token lengths; controlled request arrivals
Scheduling settings	Gemma 4 12B; context limit 4,096; at most 128 sequences and 4,096 batched tokens; chunked prefill on; prefix caching off; requested GPU-memory fraction 0.85

Study A: execution configuration

The CPU normally launches GPU operations; issuing those launches has overhead. A **CUDA graph** records a reusable execution plan so operations can be replayed with less launch overhead. We asked whether the sizes of these prepared plans matter for throughput. A plan for 32 requests may require padding when the relevant batch has 31; a matched size might avoid that extra slot. However, storing more plans consumes GPU memory. This padding-versus-memory tradeoff motivates the test; our records do not directly trace which graph is dispatched. [\[3\]](#)

We used three current models: **Gemma 4 12B**, **Qwen3.5-9B** and **Gemma 4 E4B**. Earlier Qwen2.5-1.5B and Qwen2.5-7B models served as implementation controls. This checks whether one configuration observation repeats across models; it does not test scheduling generalization.

Study B: admission under a workload change

We then asked whether a simple limit based on request size helps when prompt lengths change abruptly. The baseline is **vLLM**, the software that runs and batches requests on the GPU. We compare its normal submission path with two external admission gates: a fixed request limit and our context-budget candidate.

The idea behind a size-aware gate is to admit fewer large requests before they overwhelm resources. The danger is holding back requests that could have been processed together. A controlled burst lets us observe that tradeoff and the queue's recovery. Unlike Study A's fixed batches, requests arrive over time. Study A does not prove the cause of Study B's delays.

How this relates to existing research

Sarathi-Serve studies how to combine prefill and decode while balancing throughput and latency. SOLA uses request and system state to improve service-target attainment. These establish that scheduling already matters; they do not show that our simple rule beats a current engine. Their hardware, workloads and baselines differ from ours. [\[1\]](#) [\[4\]](#)

Exact model/data revisions, settings, traces, code, logs and raw records are retained in the [experiment inventory](#) and [W&B: project](#).

03 / STUDY A: COMPLETED

Did capture-size tuning help?

An offline, fixed-batch comparison. This tests whole-generation throughput, including prefill and host work.

Protocol. Each model used batches of 24, 31, 48, 63, 80, 95, 112 and 127 requests. Every request had 128 input and 128 output tokens. We compared default (35 capture sizes), matched (17) and coarse (9) sets, each with a maximum capture size of 256. Each configuration had three warmed timing repetitions.

Count. 8 batches x 3 settings x 3 repetitions = **72 timed calls per model**; 216 across the three models below, and 360 including the two earlier controls. Initialization and ten separate profiler passes are excluded from the timing results.

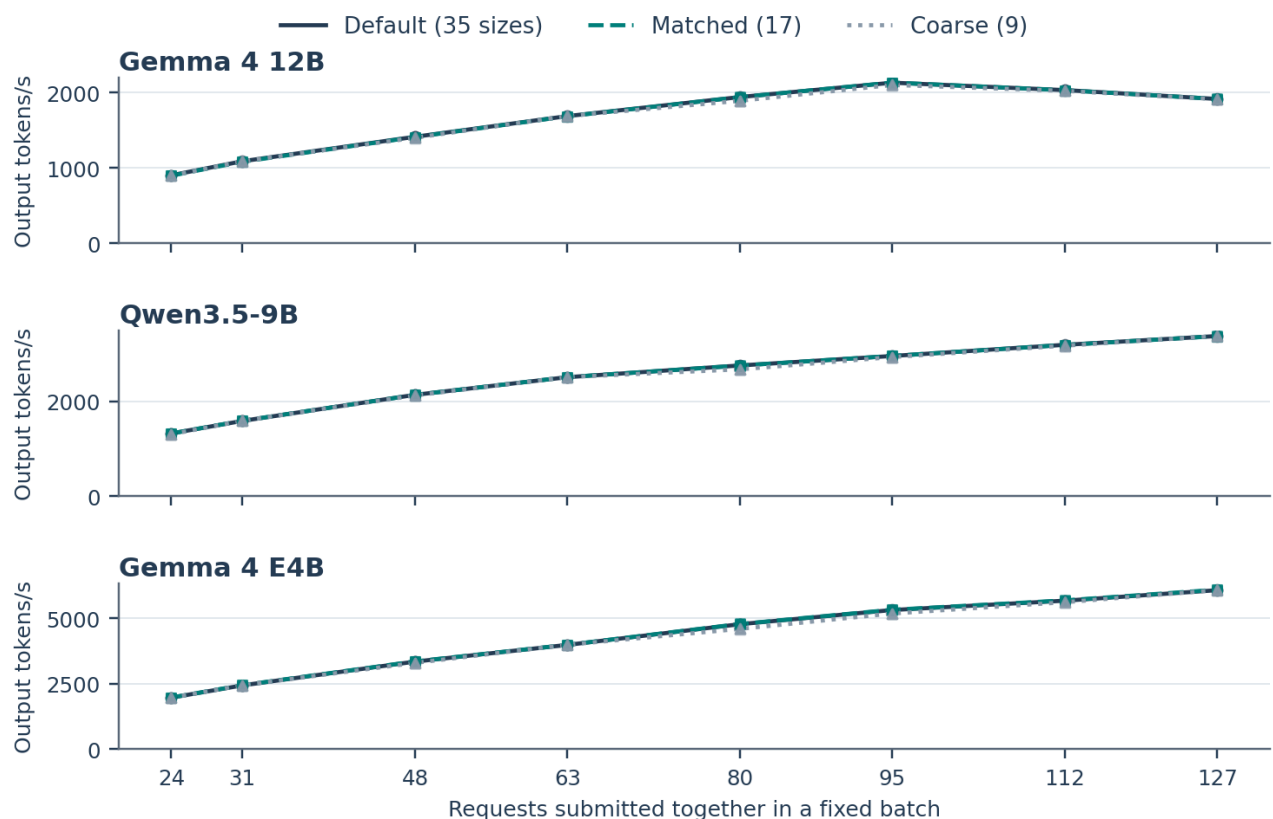


Figure 1. Our actual measurements. Dots show all three timing repetitions; lines are their means. Close points overlap. Read the horizontal axis as submitted batch size and the vertical axis as completed output tokens per second; higher is better. Compare configurations within a panel, not model answer quality across panels. [6]

What the graph shows

Matched settings differed from default by about **-0.6% to +0.3%** across the current models; coarse settings lost up to **3.8%**. These small matched differences do not establish an improvement. Each configuration's repetitions share a process, and configuration order was fixed.

Why might matching fail to help? The default already prepares 35 sizes, and graph-size overhead may be a small fraction of full generation time. Those are plausible explanations, not measured causes. The result tells us manual matching did not produce a useful gain here; it does not tell us exactly why.

For Gemma 4 12B, default throughput falls from about **2,130 to 1,914 output tokens/s** between batches 95 and 127: **10.1% lower**. Logs also reach 99.7% KV-cache occupancy, including warmup. The cache stores intermediate model state. This coincidence does not prove that cache pressure caused the drop, and cache occupancy is not GPU compute utilization.

04 / STUDY B: COMPLETED

Did our admission rule help?

Gemma 4 12B on the same A100. All 2,160 requests completed; each generated exactly 128 output tokens.

Protocol. Each saved trace had 96 short (128-token), 48 long (2,048-token), then 96 short prompts. Exponential arrival gaps averaged 8 requests/s. Three independent engine runs rotated policy order; each policy replayed the same trace within a run. Warmup was excluded and requests drained between trials. [5]

Policies. Default submits directly to vLLM. Fixed-32 admits at most 32 unfinished requests in arrival order. Context budget reserves input length + 128 tokens per admitted request under a 20,000-token cap, choosing the oldest fitting request with two-second age protection. This is an estimate, not measured KV memory.

Targets: time to first token (TTFT) at most 2 s, then mean time per output token at most 100 ms. Waiting outside the engine counts.

SLO goodput = requests meeting both targets / full replay-and-drain seconds. The frozen pass rule required at least 5% more goodput than both controls in every paired run, without worse p99 TTFT or p99 stream gap (time between streamed updates).

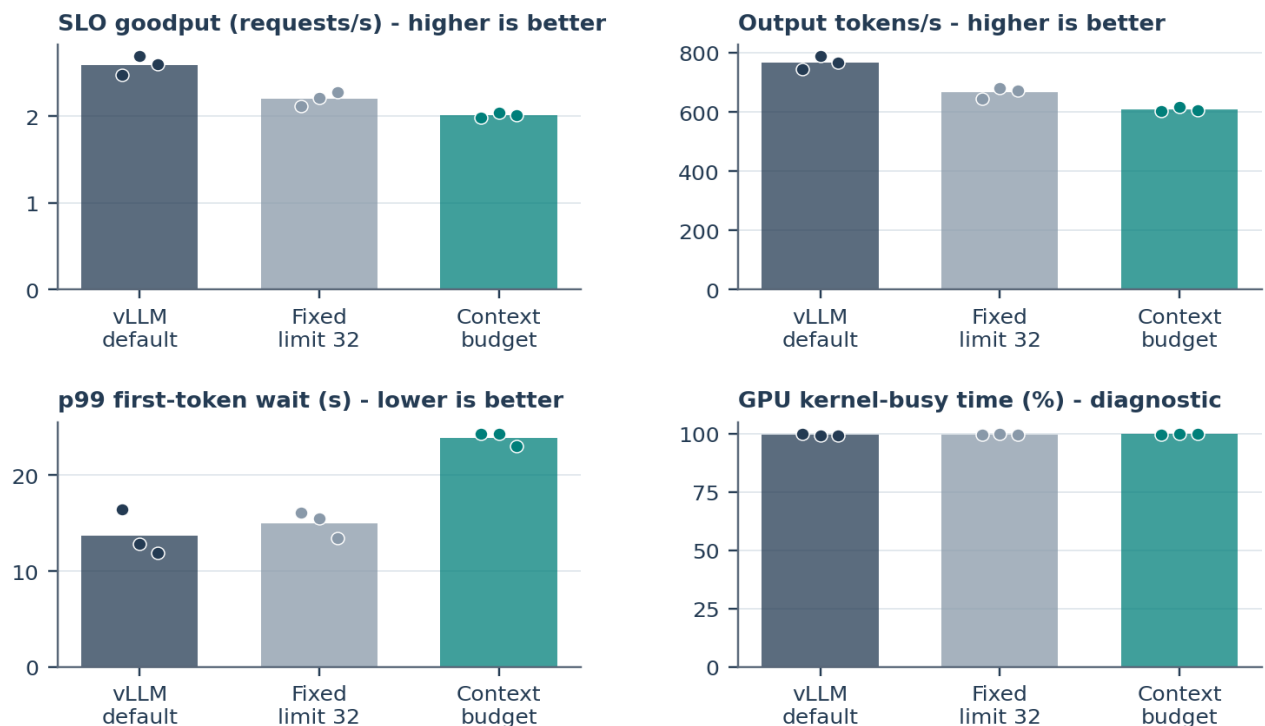


Figure 2. Our actual measurements. Dots are three independent engine runs; bars are means. Higher top panels are better; lower first-token wait is better. “p99” is the 99th percentile, describing the slow end of the distribution. GPU busy time is a diagnostic, not our success criterion. [5]

Mean across 3 runs	Goodput (req/s)	Output (tokens/s)	p99 TTFT (s)	GPU busy (%)
vLLM default	2.588	766.6	13.70	99.46
Fixed limit 32	2.202	665.3	14.99	99.67
Context budget	2.013	608.3	23.89	99.78

The candidate failed all six paired comparisons. Goodput was 20.0-24.0% lower than default vLLM, even though mean GPU busy time was slightly higher. The p99 columns average per-run p99 values, not pooled percentiles. No statistical-significance or answer-quality equivalence claim is made.

05 / INTERPRETATION AND NEXT STEP

What can we honestly conclude?

The experiments reveal a reproducible problem in this workload and reject the tested rule. They do not deliver an improved scheduler.

The most relevant finding is poor burst recovery

A later analysis of the saved records found **96/96 initial short requests met the targets, but 0/96 later short requests did**, in every policy and repetition. Only 4-9 of the 48 long requests passed. The context-budget candidate needed about 20-22 seconds after the final arrival to finish, versus 10-12 seconds for default vLLM. [Phase records](#).

The gate did not rescue the later short requests. Similar numbers passed across policies, but the candidate took longer to finish all work, lowering goodput through its time denominator. This finite burst at one arrival rate does not measure sustainable capacity or show that scheduling can remove the misses at this load.

Is this a sensible way to investigate?

The comparison is useful, but the gate may be too restrictive. Each long request reserves $2,048 + 128 = 2,176$ **estimated tokens**, so a 20,000-token budget admits at most **nine all-long requests**, versus 32 in the fixed control. This can prevent useful batching while requests wait outside the engine. The GPU can remain busy with the admitted work even as total completion slows. These are admitted requests, not a count of simultaneous GPU executions.

That mechanism is a hypothesis, not an isolated causal result: the candidate also changes request ordering and age protection. Its two-second age rule activates when the first-token allowance is already used up, and its reservation is neither measured processing cost nor actual live KV memory. The next comparison must separate these decisions.

The hardware was busy almost continuously, but achieved processing-unit efficiency was not measured. Scheduling was tested on only one model, and output hashes differed across policies. We cannot claim GPU compute-efficiency gains, generalization to other models or equal answer quality.

One next experiment: test adaptation fairly

On separate calibration traces, measure waiting and processing cost and tune a simple fixed limit. Freeze that baseline and a candidate using cost and time remaining before deadlines. Compare both with default vLLM on unseen burst traces at several predeclared arrival rates, using balanced independent engine runs. Keep fairness rules matched and include a version with adaptation removed to isolate its effect.

Evaluate goodput, throughput, recovery after the burst and long-request tail latency; account for every request and any drops. Profile separately if claiming hardware-efficiency changes. The result may support adaptation, reject it, or show the load exceeds what the hardware can serve within those targets.

Sources and reproducible records

- [1] Agrawal et al. Sarathi-Serve. OSDI 2024. Prefill/decode scheduling and latency-throughput tradeoffs.
 - [2] NVIDIA. NVML utilization definition. Kernel-busy time is not achieved compute efficiency.
 - [3] vLLM. CUDA Graphs design documentation. Capture/replay and dispatch behavior.
 - [4] Hong et al. SOLA. MLSys 2025. Request- and system-state-aware SLO scheduling.
 - [5] Our scheduling study. Protocol, traces, request records, telemetry and per-run summaries.
 - [6] Our capture study. Per-model measurements, means and raw-data provenance.
- [Repository](#) | [Full project bibliography](#) | [Complete inventory](#) | [W&B: records](#)